GESTIÓN DE PROCESOS- LAB INTEGRADOR

En esta actividad integramos y aplicamos los conceptos teóricos vistos en la materia. Vamos a analizar procesos reales del sistema, trazar diagramas de estados y detectar comportamientos sospechosos desde la perspectiva de la ciberseguridad mediante:

- Observar procesos reales
- Trazar diagramas de estados
- Implementar fork() y wait()
- Detectar procesos sospechosos

CONSIGNAS

1- Explorador de procesos parte 2

Abrí una terminal en Linux (o WSL). Ejecutá los siguientes comandos y respondé las preguntas.

```
$ ps -A
$ ps -A | grep bash
$ ps -eaf | head -20
$ top  (presionar 'q' para salir)
$ kill -l
$ pstree -p | head -20
$ sudo cat /proc/[PID]/maps
```

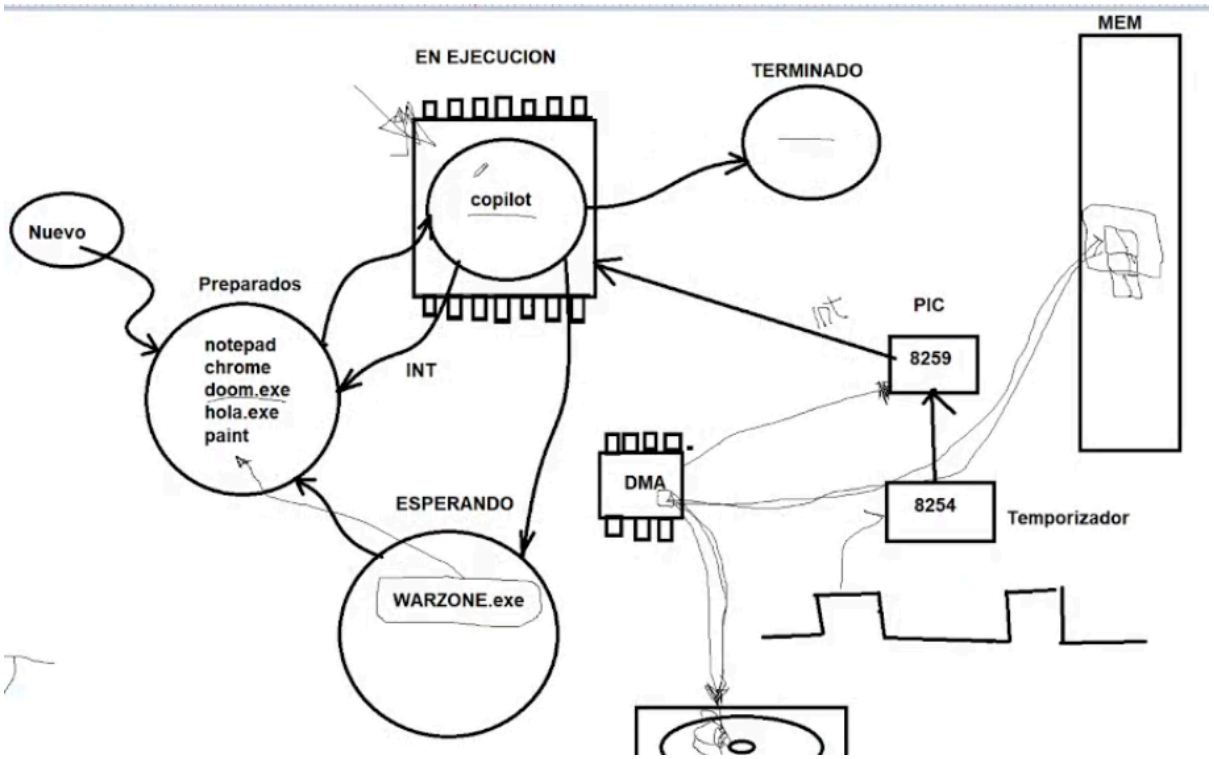
PREGUNTAS

A. ¿Cuántos procesos están activos?
B. ¿Cuál es el PID del proceso bash? (la bash es la consolita)
C. ¿Qué proceso tiene el PID más bajo?
D. ¿Qué proceso tiene más tiempo de CPU?
E. ¿Qué señal termina un proceso (kill)?
F. ¿Para qué sirve el comando `sudo cat /proc/[PID]/maps`?

NOTA: Podés usar el comando del LAB anterior para ver el PCB pero también usar ps -eaf para ver el proceso padre (PPID) de cada proceso y construir mentalmente el árbol de procesos (o sea el "árbol genealógico" de los procesos).

2- Diagrama de estados

Realizar un diagrama de estados del proceso con las transiciones correctas. Por el momento trabajamos con el diagrama simplificado, cuando veamos swap le agregamos el estado "suspendido". El diagrama pueden encontrarlo en el material de la cátedra.



Obra de arte creada por Leandro Robles

NOTA: Si no vieron en clase lo que es el DMA, les dejo este videito cortito que explica muy bien lo que es (básicamente el "che pibe del procesador"). Tampoco es necesario profundizar tanto, simplemente entender su función: https://www.youtube.com/watch?v=-58fmXP5xQs

Planificadores (schedulers).

- Largo plazo → Decide **qué procesos ingresan al sistema** | Nuevo → Preparado

  Función principal

  - Selecciona procesos de la cola
  - Los carga en memoria
  - Define cuántos procesos habrá en el sistema

- Mediano plazo → Gestiona procesos **suspendidos/intercambiados(swap)** | Preparado → Suspendido
  Suspendido → Preparado

  Función principal

  - Libera memoria (swap)
  - Reduce carga del sistema
  - Evita saturación

- Corto plazo → Decide **qué proceso usa la CPU** | Preparado → Ejecutando

  Función principal

  - Selecciona el próximo proceso a ejecutar
  - Aplica algoritmos (FIFO, prioridades, etc.)

ZOMBIE

Es un proceso que finalizó su ejecución, pero mantiene una entrada en la tabla de procesos (la listita de procesos que maneja el sistema operativo) porque su padre (ver PPID) no leyó su estado de salida (diagrama de estados, recuerden que lo vimos en el lab anterior cuando vimos la versión “humana” del PCB). No consumen CPU ni memoria, pero si se acumulan, agotan el ID de proceso (PID) lo que impide que el sistema inicie nuevas aplicaciones o que el sistema empiece a fallar.

PREGUNTAS

A. ¿Qué transición genera el planificador de corto plazo?
B. ¿En qué estado queda un proceso zombie?
C. ¿Qué evento fuerza la transición EJECUTANDO → PREPARADO?

3- Programación con fork()

1. Compilá y ejecutá. En el directorio donde se encuentra tu archivo:
Usá: `gcc fork_actividad.c -o fork_act && ./fork_act`

2. ¿Cuántos procesos hay?
¿Cuántos hijos crea el bucle? Dibujá el árbol de procesos.

3. Modificá el código
Cambiá el for de 3 a 5. ¿Cambia el resultado? ¿Por qué?

4. Eliminá el wait()
¿Qué pasa si comentás el while(wait(...))? Verificá con: ps -eaf | grep defunct

5. extra
¿Podés hacer que cada hijo imprima su número de iteración (i) correctamente?

CASO REAL: Ciberseguridad y Gestión de Procesos

ESCENARIO: Sos analista de seguridad en la UNAHUR. El sistema de monitoreo alerta que la PC de un usuario tiene un uso de CPU inusualmente alto. Tu misión: analizar los procesos activos y determinar si hay actividad maliciosa. (parecido al LAB anterior)

Ejecutar el script simulador (no copies el $)

```
$ chmod +x lab_procesos.sh
$ ./simulador_procesossospechosos.sh
```

Podés revisar con top o htop si ya funciona correctamente. Lo que hace este script es simular procesos sospechosos y zombies para que trabajes de una manera más realista. Esto es simplemente para jugar un poco.

ALERTAS

⚠️ **Proceso sin nombre visible** **Un proceso aparece como [alumno/u:1] pero usa 40% de CPU. Los procesos legítimos del kernel rara vez consumen tanto.**
⚠️ **Proceso zombie masivo** **Se detectan 50+ procesos en estado 'Z' (zombie). Puede indicar un exploit que genera hijos sin control.**
⚠️ **PID muy bajo, proceso extraño** **Un proceso con PID 9 se llama 'systmd' (con d al final). El demonio (programita que ejecuta alguna tarea importante en segundo plano) real del sistema es 'systemd'. Los PID bajos generalmente son del sistema (como INIT)**

¡A practicar!

A. Listar todo con CPU % ordenado → `ps -eo pid,ppid,cmd,%cpu --sort=-%cpu | head`
B. Ver procesos zombie → `ps -A -ostat,ppid,pid,cmd | grep "^Z"`
C. Ver árbol completo de procesos → `pstree -p`
D. Inspeccionar un proceso específico → `cat /proc/[PID]/status` luego ingresá en la consola `ls -la /proc/[PID]/exe`

Ejercicio de Análisis Forense

Con el resultado del comando que utilizamos en el ejercicio anterior (ps -eo pid,ppid,stat,cmd,%cpu --sort=-%cpu) vamos a observar y hacernos algunas preguntas:

1. ¿Por qué me sirve poder diferenciar un programa de un proceso?
2. ¿Para qué me sirve poder identificar la relación que existe entre procesos (padre/hijo)?
3. ¿Por qué me ayuda conocer los estados de un proceso?
4. ¿Por qué es relevante entender cómo funcionan los planificadores?
5. ¿Para qué me sirve conocer las señales que maneja el sistema operativo?
6. Documentar todo lo que sea posible y basarse en la teoría de la materia.

ANEXO

En este apartado les adjunto un anexo de un laboratorio que resolví hace unos meses donde se aplican conocimientos adquiridos en esta materia. Este anexo es netamente informativo y no se espera que entiendan la totalidad de lo expuesto, sino simplemente evidenciar que los contenidos que están aprendiendo, tienen utilidades diferentes según el contexto en que se apliquen y no lo vean como algo únicamente teórico.

Fragmento del laboratorio Oski- CyberDefenders

6. Acciones post-exfiltración (Evasión)

Una vez finalizado el robo y la exfiltración de datos, el malware ejecuta acciones para **dificultar el análisis forense.**

Uso de procesos hijos

El malware inicia un proceso hijo:

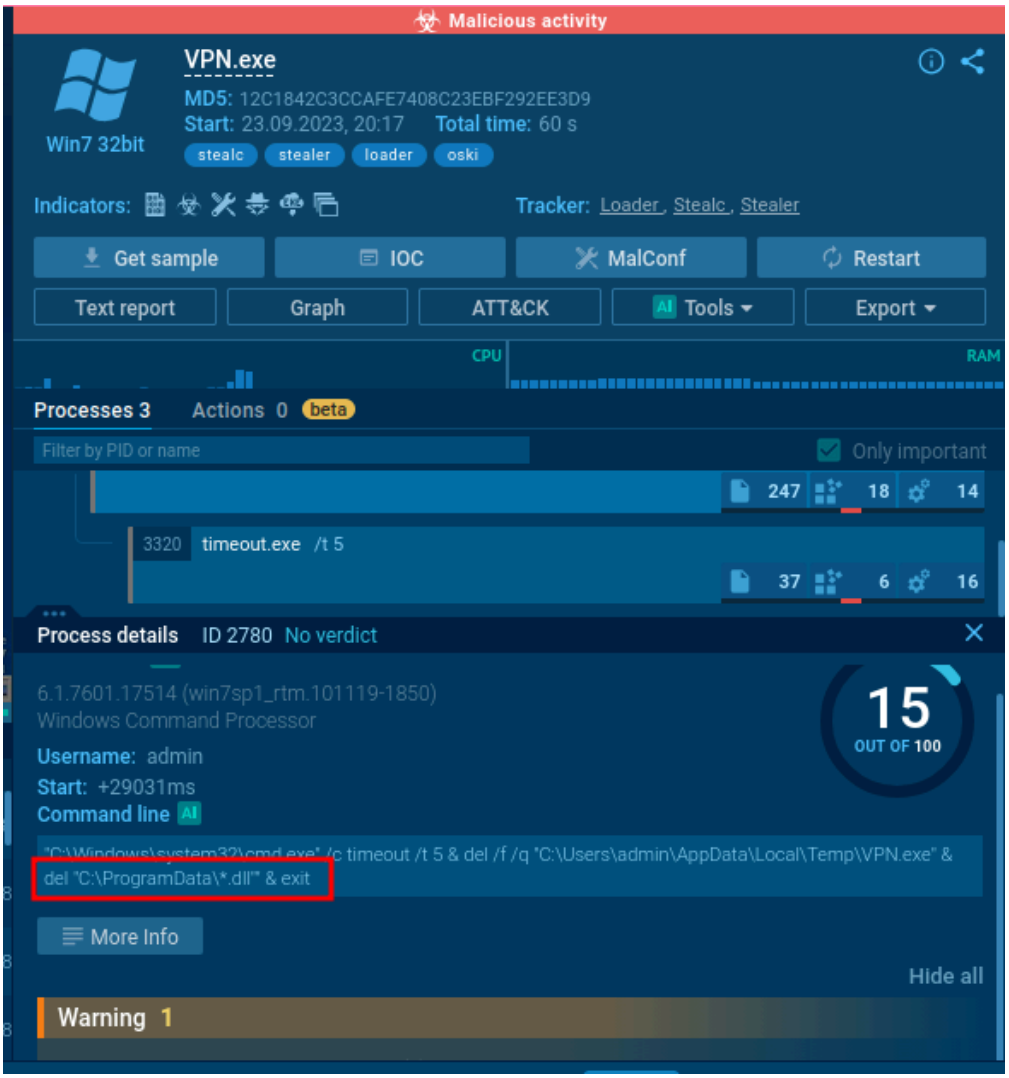
- `cmd.exe`

Este proceso es utilizado para ejecutar comandos de **auto-eliminación (self-delete).**

Directorios afectados

Se observan dos acciones principales:

1. Eliminación del ejecutable principal:
    - `C:\Users\admin\AppData\Local\Temp\VPN.exe`
2. Eliminación masiva de librerías dinámicas:
    - `C:\ProgramData\` (borrado de `*.dll`)



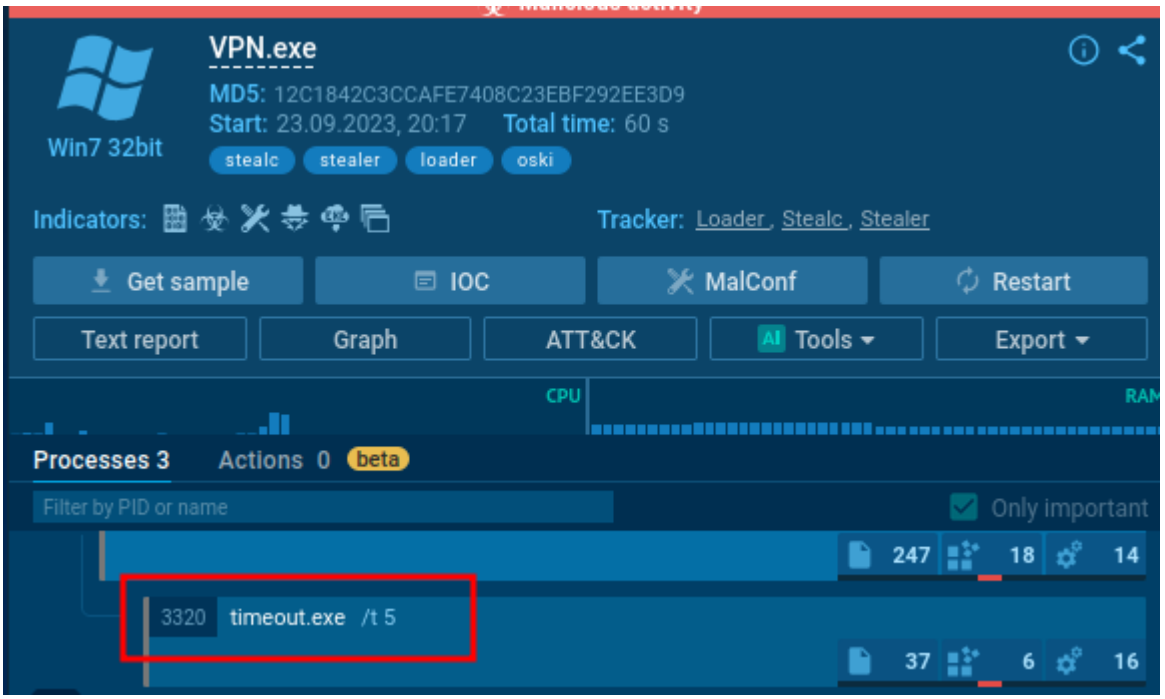
7.Temporización

El process tree indica un **delay de 5 segundos** antes de ejecutar el self-delete.

¿Por qué es importante?

Estas acciones buscan:

- Eliminar rastros de la ejecución
- Reducir indicadores en disco
- Dificultar investigaciones posteriores



Conclusión

El laboratorio presenta un escenario realista de **infección por malware tipo stealer**, distribuido mediante ingeniería social.
El análisis permitió reconstruir la cadena completa del ataque, desde la ejecución inicial hasta las acciones de evasión post-exfiltración.